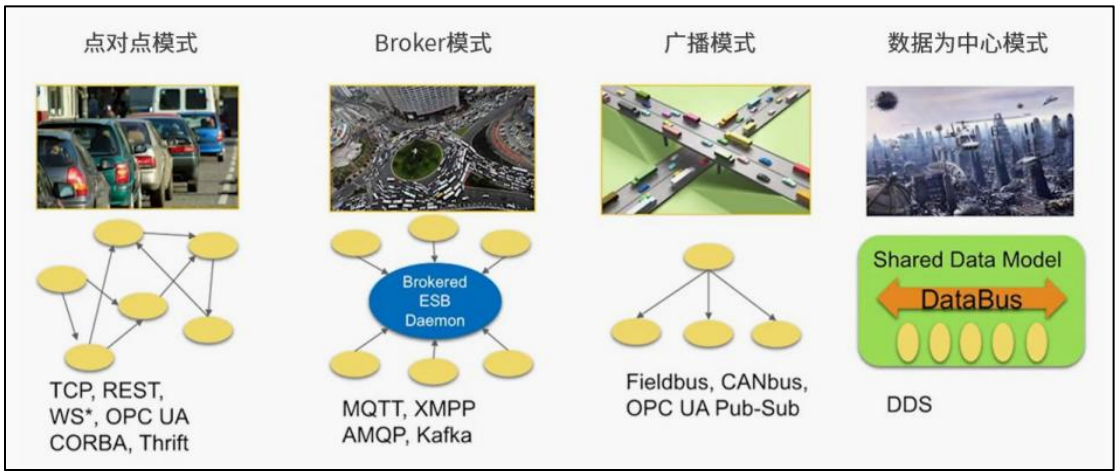
14.ROS2 DDS 说明

1.DDS 简介

DDS 的全称是 Data Distribution Service，也就是数据分发服务，2004 年由对象管理组织 OMG 发布和维护，是一套专门为实时系统设计的数据分发/订阅标准，最早应用于美国海军，解决舰船复杂网络环境中大量软件升级的兼容性问题，现在已经成为强制标准。

DDS 强调以数据为中心，可以提供丰富的服务质量策略，以保障数据进行实时、高效、灵活地分发，可满足各种分布式实时通信应用需求。

在前面课程中学习的话题、服务、动作，他们底层通信的具体实现过程，都是靠 DDS 来完成的，它相当于 ROS 机器人系统中的神经网络。常见的通信模型有以下四种：



1） 点对点模型，许多客户端连接到一个服务端，每次通信时，通信双方必须建立一条连接。当通信节点增多时，连接数也会增多。而且每个客户端都需要知道服务器的具体地址和所提供的服务，一旦服务器地址发生变化，所有客户端都会受到影响。

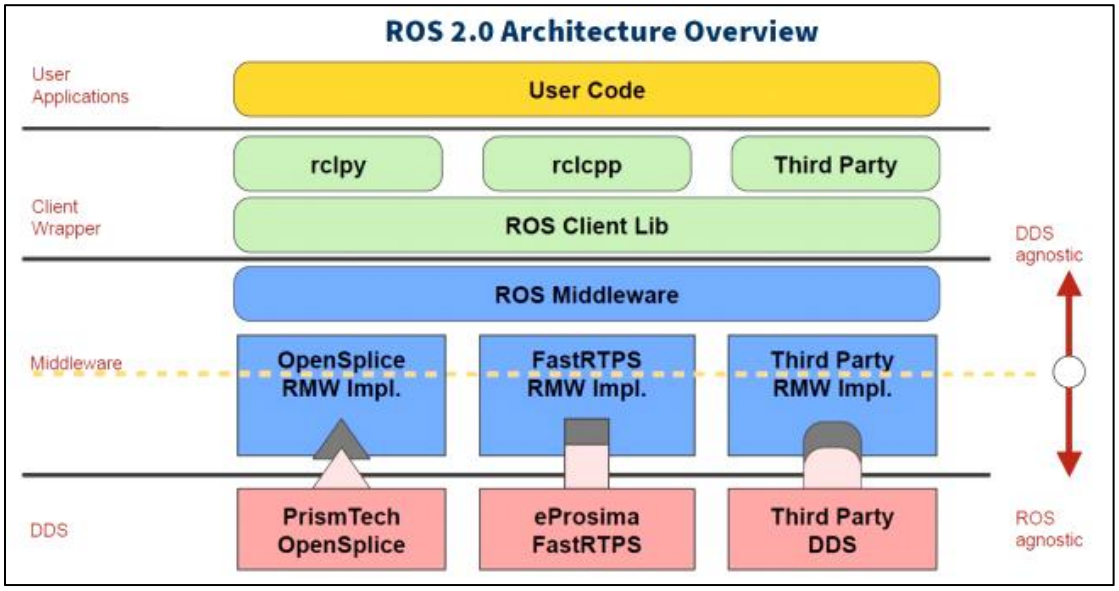
2） Broker 模型，针对点对点模型进行了优化，由 Brocker 集中处理所有人的请求，并进一步找到真正能响应该服务的角色。这样客户端就不用关心服务器的具体地址了。不过问题也很明显，Broker 作为核心，它的处理速度会影响所有节点的效率，当系统规模增长到一定程序，Broker 就会成为整个系统的性能瓶颈。更麻烦的是，如果 Broker 发生异常，可能导致整个系统都无法正常运转。之前的 ROS1 系统，使用的就是类似这样的架构。

3） 广播模型，所有节点都可以在通道上广播消息，并且节点都可以收到消息。这个模型解决了服务器地址的问题，而且通信双方也不用单独建立连接，但是广播通道上的消息太多了，所有节点都必须关心每条消息，其实很多是和自己没有关系的。

4） 以数据为中心的 DDS 模型，这种模型与广播模型有些类似，所有节点都可以在 DataBus 上发布和订阅消息。但它的先进之处在于，通信中包含了很多并行的通路，每个节点可以只关心自己感兴趣的消息，忽略不感兴趣的消息，有点像是一个旋转火锅，各种好吃的都在这个 DataBus 传送，我们只需要拿自己想吃的就行，其他的和我们没有关系。

2.DDS 在 ROS2 中的应用

DDS 在 ROS2 系统中的位置至关重要，所有上层建设都建立在 DDS 之上。在这个 ROS2 的架构图中，蓝色和红色部分就是 DDS。



在 ROS 的四大组成部分中，由于 DDS 的加入，大大提高了分布式通信系统的综合能力，这样我们在开发机器人的过程中，就不需要纠结通信的问题，可以把更多时间放在其他部分的应用开发上。

3.质量服务策略 Qos

DDS 中的基本结构是 Domain，Domain 将各个应用程序绑定在一起进行通信。DDS 中另外一个重要特性就是质量服务策略：Qos。

Qos 是一种网络传输策略，应用程序指定所需要的网络传输质量行为，Qos 服务实现这种行为要求，尽可能地满足客户对通信质量的需求，可以理解为数据提供者和接收者之间的合约。策略如下：

◆ DEADLINE 策略，表示通信数据必须要在每次截止时间内完成一次通信；

◆ HISTORY 策略，表示针对历史数据的一个缓存大小；

◆ RELIABILITY 策略，表示数据通信的模式，配置成 BEST_EFFORT，就是尽力传输模式，网络情况不好的时候，也要保证数据流畅，此时可能会导致数据丢失，配置成 RELIABLE，就是可信赖模式，可以在通信中尽量保证图像的完整性，我们可以根据应用功能场景选择合适的通信模式；

◆ DURABILITY 策略，可以配置针对晚加入的节点，也保证有一定的历史数据发送过去，可以让新节点快速适应系统。

4.命令行中配置 DDS

1） 使用 Nomachine 远程连接工具连接 Jetson Orin Nano 主板，然后双击桌面的 

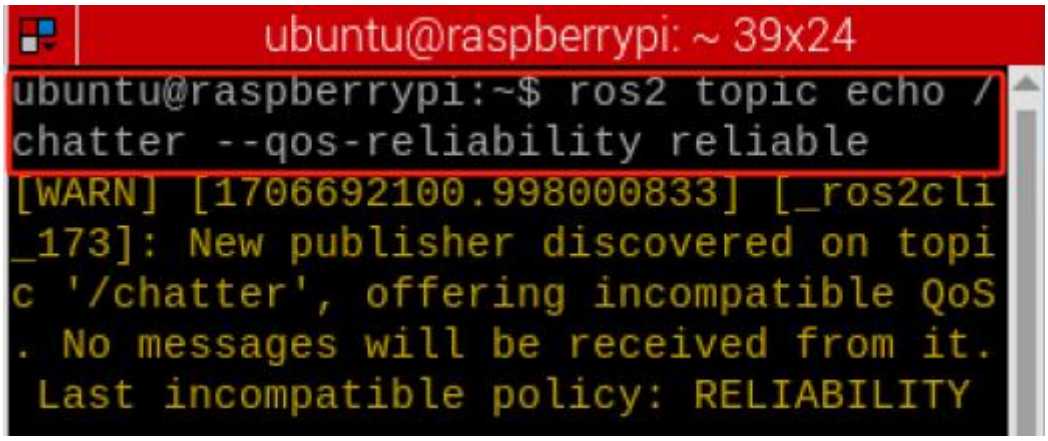
，输入指令

“**ros2 topic pub /chatter std_msgs/msg/Int32 "data: 42" --qos-reliability best_effort**”，发布一个名为"/chatter"的话题，使用 std_msgs/msg/Int32 消息类型，并发送一个数据为 42 的整数消息。通过添加"--qos-reliability best_effort"选项，指定了发布者使用最佳尽力传输（best effort）的可靠性。

```
ubuntu@ubuntu:~$ ros2 topic pub /chatter std_msgs/msg/Int32 "data: 42" --qos-re
liability best_effort
```

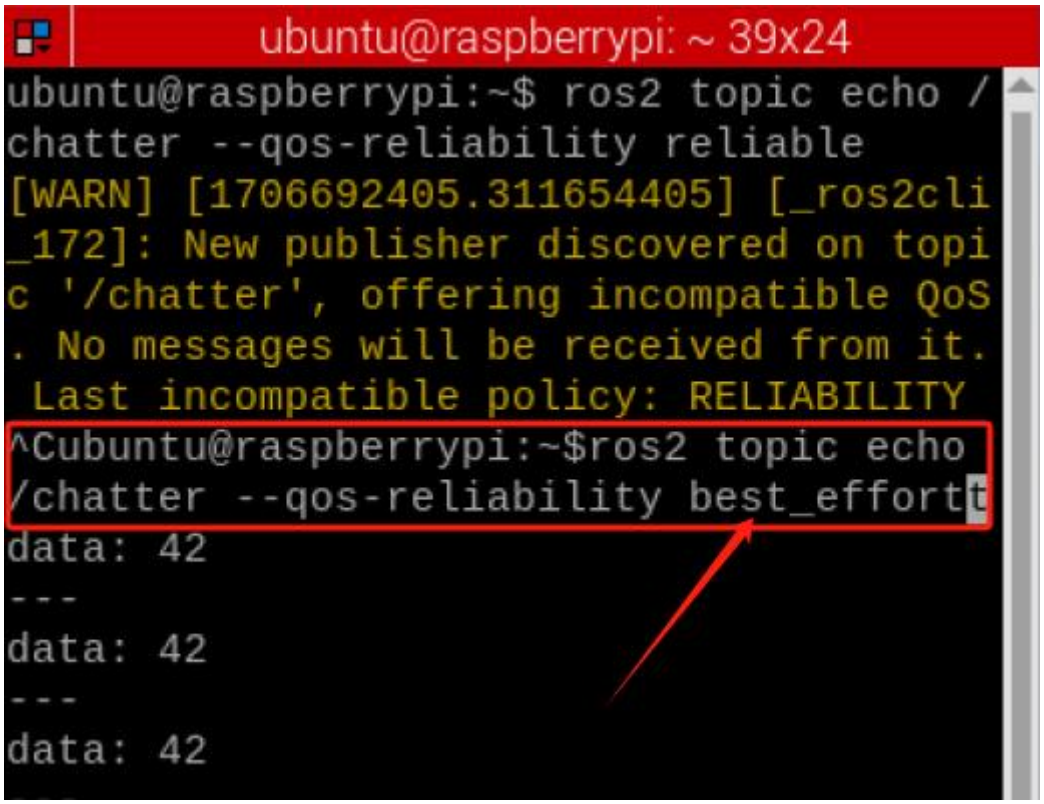
2） 然后新建一个命令行终端，输入指令

“**ros2 topic echo /chatter --qos-reliability reliable**”指令，订阅一个名为"/chatter"的话题，并打印出接收到的消息。通过添加"--qos-reliability reliable"选项。



使用 reliable 模型订阅同一话题，无法实现数据通信，如果修改为同样的 best_effort，才能实现数据传输。

3） 输入“**ros2 topic echo /chatter --qos-reliability best_effort**”指令，订阅一个名为"/chatter"的话题，并打印出接收到的消息。通过添加"--qos-reliability reliable"选项



修改为同样的 best_effort，才能实现数据传输。

5.DDS 编程示例

5.1 创建发布者

1） 使用 Nomachine 远程连接工具连接 Jetson Orin Nano 主板，然后双击桌面的 

，输入指令“**cd ros2_ws/src/**”指令，切换到 ros2_ws 工作空间的 src 文件夹中。

```
ubuntu@ubuntu:~$ cd ros2_ws/src/
ubuntu@ubuntu:~/ros2_ws/src$
```

2） 输入指令

“**ros2 pkg create DDS_qos_demo --build-type ament_python --dependencies rclpy**”并按下回车，创建一个名为“**DDS_qos_demo**”的功能包，添加依赖关系 rclpy。

```
ubuntu@ubuntu:~/ros2_ws/src$ ros2 pkg create DDS_qos_demo --build-type ament_py
thon --dependencies rclpy
going to create a new package
package name: DDS_qos_demo
```

3） 输入“**cd DDS_qos_demo/DDS_qos_demo/**”指令，切换到 DDS_qos_demo 功能包下。

```
ubuntu@ubuntu:~/ros2_ws/src$ cd DDS_qos_demo/DDS_qos_demo/
ubuntu@ubuntu:~/ros2_ws/src/DDS_qos_demo/DDS_qos_demo$
```

1） 输入指令“**gedit DDS_qos_pub.py**”编辑程序，复制下面程序,使用快捷键“ctrl+s”保存并退出。

```
ubuntu@ubuntu:~/ros2_ws/src/DDS_qos_demo/DDS_qos_demo$ gedit DDS_qos_pub.py
```

```
import rclpy  # 导入 rclpy 模块

from rclpy.node import Node  # 导入 Node 类



from std_msgs.msg import String  # 导入 String 消息类型

from rclpy.qos import QoSProfile, QoSReliabilityPolicy, QoSHistoryPolicy
  # 导入 QoSProfile 和 QoSReliabilityPolicy、QoSHistoryPolicy 类
```

```
class MinimalPublisher(Node):  # 定义一个继承自 Node 类的 MinimalPublisher 类


    def __init__(self):
        super().__init__('minimal_publisher')  # 调用父类构造函数初始化节点
        qos_profile = QoSProfile(  # 创建 QoSProfile 对象
            reliability=QoSReliabilityPolicy.RELIABLE,  # 设置可靠性策略为
RELIABLE
            history=QoSHistoryPolicy.KEEP_LAST,  # 设置历史策略为 KEEP_LAST
            depth=1  # 设置深度为 1
        )
        self.publisher_ = self.create_publisher(String, 'topic', qos_prof
ile)  # 创建发布者对象
        timer_period = 0.5  # 设置定时器回调的时间间隔为 0.5 秒
        self.timer = self.create_timer(timer_period, self.timer_callback)
  # 创建定时器，设置回调函数为 timer_callback
        self.i = 0


    def timer_callback(self):
        msg = String()  # 创建 String 消息类型的对象
        msg.data = 'Hello World: %d' % self.i  # 设置消息的数据
        self.publisher_.publish(msg)  # 发布消息
        self.i += 1  # 自增计数器
```

```
def main(args=None):
    rclpy.init(args=args)  # 初始化 ROS 节点

    minimal_publisher = MinimalPublisher()  # 创建 MinimalPublisher 对象

    rclpy.spin(minimal_publisher)  # 进入主循环

    minimal_publisher.destroy_node()  # 销毁节点
    rclpy.shutdown()  # 关闭 ROS

if __name__ == '__main__':
    main()
```

4） 输入指令“**chmod +x DDS_qos_pub.py**”，并按下回车，为保存的 topic_pub.py 赋予可执行权限。

```
ubuntu@ubuntu:~/ros2_ws/src/DDS_qos_demo/DDS_qos_demo$ chmod +x DDS_qos_pub.py
```

5.2 创建发布者

1） 输入指令“**gedit DDS_qos_sub.py**”编辑程序，复制下面程序,使用快捷键“ctrl+s”保存并退出。

```
ubuntu@ubuntu:~/ros2_ws/src/DDS_qos_demo/DDS_qos_demo$ gedit DDS_qos_sub.py
```

```
import rclpy  # 导入 rclpy 模块
```

```
from rclpy.node import Node  # 导入 Node 类


from std_msgs.msg import String  # 导入 String 消息类型

from rclpy.qos import QoSProfile, QoSReliabilityPolicy, QoSHistoryPolicy
  # 导入 QoSProfile 和 QoSReliabilityPolicy、QoSHistoryPolicy 类


class MinimalSubscriber(Node):  # 定义一个继承自 Node 类的 MinimalSubscriber
类


    def __init__(self):

        super().__init__('minimal_subscriber')  # 调用父类构造函数初始化节
点

        qos_profile = QoSProfile(  # 创建 QoSProfile 对象

            reliability=QoSReliabilityPolicy.RELIABLE,  # 设置可靠性策略为
RELIABLE

            history=QoSHistoryPolicy.KEEP_LAST,  # 设置历史策略为 KEEP_LAST

            depth=1  # 设置深度为 1

        )

        self.subscription = self.create_subscription(String, 'topic', sel
f.listener_callback, qos_profile)  # 创建订阅者对象，并设置回调函数为 listen
er_callback


    def listener_callback(self, msg):
```

```
        self.get_logger().info('I heard: "%s"' % msg.data)  # 打印接收到的消息内容



def main(args=None):
    rclpy.init(args=args)  # 初始化 ROS 节点

    minimal_subscriber = MinimalSubscriber()  # 创建MinimalSubscriber对象

    rclpy.spin(minimal_subscriber)  # 进入主循环

    minimal_subscriber.destroy_node()  # 销毁节点
    rclpy.shutdown()  # 关闭 ROS

if __name__ == '__main__':
    main()
```

2） 输入指令“**chmod +x DDS_qos_sub.py**”，并按下回车，为保存的 topic_sub.py 赋予可执行权限。

```
ubuntu@ubuntu:~/ros2_ws/src/DDS_qos_demo/DDS_qos_demo$ chmod +x DDS_qos_sub.py
```

3.setup.py 文件设置

setup.py 文件是定义一个 ROS2 包的远数据和构建配置，提供了包的元数据、依赖关

系、构建配置和安装逻辑等信息，可以帮助开发者正确构建、安装和使用 ROS2 包。需要将 **topic_pub.py** 和 **topic_sub.py** 的程序入口写到 setup.py 文件中。

1） 输入“**cd ..**”指令，切换到上一级目录。

```
ubuntu@ubuntu:~/ros2_ws/src/DDS_qos_demo/DDS_qos_demo$ cd ..
```

2） 输入“**gedit setup.py**”指令，并按下回车，打开 setup.py 文件，复制下面程序,使用快捷键“ctrl+s”保存并退出。

```
ubuntu@ubuntu:~/ros2_ws/src/DDS_qos_demo$ gedit setup.py
```

```
'DDS_qos_pub = DDS_qos_demo.DDS_qos_pub:main',

'DDS_qos_sub = DDS_qos_demo.DDS_qos_sub:main'
```

```
15      zip_safe=True,
16      maintainer='ubuntu',
17      maintainer_email='ubuntu@todo.todo',
18      description='TODO: Package description',
19      license='TODO: License declaration',
20      tests_require=['pytest'],
21      entry_points={
22          'console_scripts': [
23              'DDS_qos_pub = DDS_qos_demo.DDS_qos_pub:main',
24              'DDS_qos_sub = DDS_qos_demo.DDS_qos_sub:main'
25          ],
26      },
27  )
```

6.编译运行

1） 使用 Nomachine 远程连接工具连接 Jetson Orin Nano 主板，然后双击桌面的 

，输入指令“**cd ~/ros2_ws/**”指令，切换到工作空间的目录。

```
ubuntu@ubuntu:~$ cd ~/ros2_ws/
ubuntu@ubuntu:~/ros2_ws$
```

2） 输入“**colcon build**”指令，并按下回车，编译工作空间中的功能包。

```
ubuntu@ubuntu:~/ros2_ws$  colcon build
```

3） 再输入指令“**source ./install/setup.bash**”，并按下回车，让环境变量生效。

```
ubuntu@ubuntu:~/ros2_ws$ source ./install/setup.bash
```

4） 输入指令“**ros2 run DDS_qos_demo DDS_qos_pub**”，并按下回车，启动 DDS_qos_pub 话题发布节点。

```
ubuntu@ubuntu:~/ros2_ws$ ros2 run DDS_qos_demo DDS_qos_pub
```

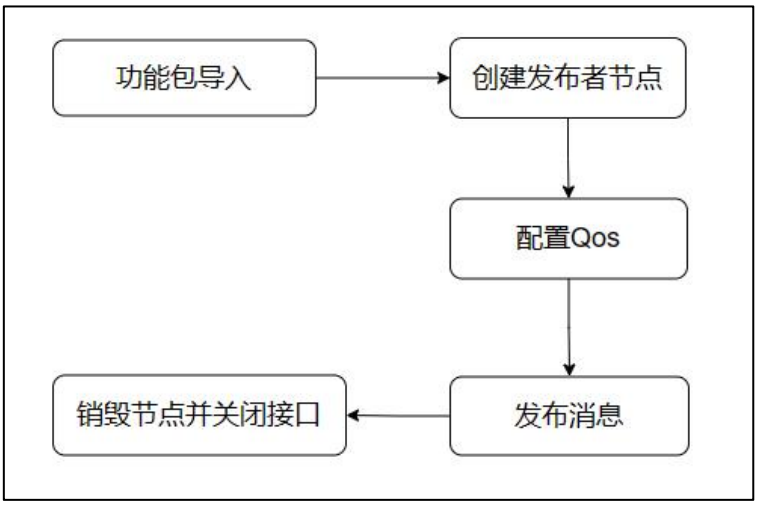
5） 用 Nomachine 远程连接工具连接 Jetson Orin Nano 主板，然后双击桌面的 

，参考步骤 1）-3）新建一个新的终端窗口并且更新当前的终端环境之后，输入指令“**ros2 run DDS_qos_demo DDS_qos_sub**”，并按下回车，启动 DDS_qos_sub 话题订阅节点。

7.程序分析

7.1 话题发布

根据实现的效果，梳理程序的过程逻辑，如下图所示：



创建了一个 MinimalPublisher 节点类，在构造函数中创建了一个 QosProfile 对象，配置发布为可靠模式，保证最后一次或第一条消息，并设置缓冲深度为 1。然后创建了一个 0.

5 秒周期的定时器对象。定时器回调函数 timer_callback 中,会构造 String 类型消息,随着循环计数器 i 增加,改变消息内容。并通过发布者对象 publisher_发布这条消息字符串。主函数首先初始化 ROS 节点环境,然后创建节点实例,进入主循环来驱动定时任务的执行。定时任务使发布内容能循环发送。最后清理节点资源。

◆ 主函数

```
def main(args=None):
    rclpy.init(args=args)  # 初始化ROS节点

    minimal_publisher = MinimalPublisher()  # 创建MinimalPublisher对象

    rclpy.spin(minimal_publisher)  # 进入主循环

    minimal_publisher.destroy_node()  # 销毁节点
    rclpy.shutdown()  # 关闭ROS
```

首先调用 rclpy.init()对 ROS2 Python 接口进行初始化，然后实例化 MinimalPublisher()，然后在 ROS2 节点的事件循环执行 minimal_publisher。

◆ MinimalPublisher 类

```
class MinimalPublisher(Node):  # 定义一个继承自Node类的MinimalPublisher类

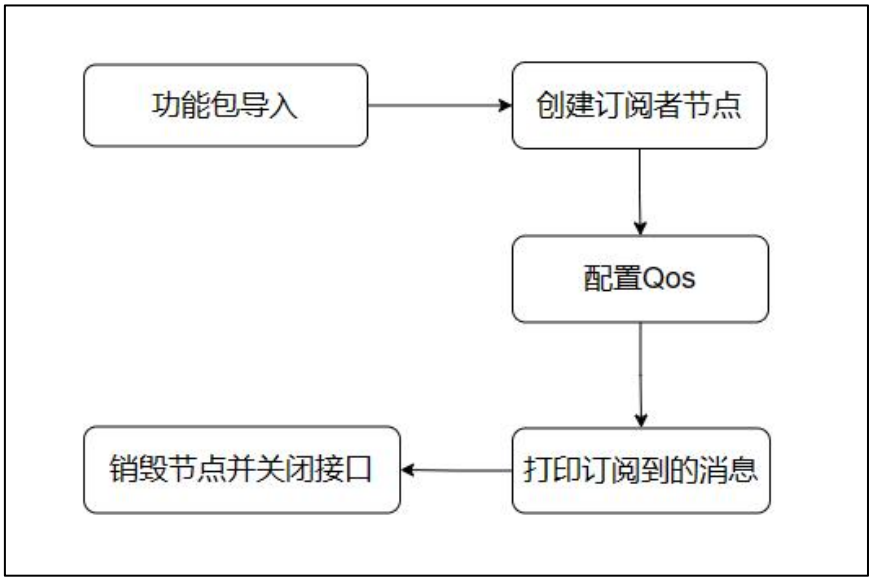
    def __init__(self):
        super().__init__('minimal_publisher')  # 调用父类构造函数初始化节点
        qos_profile = QoSProfile(  # 创建QoSProfile对象
            reliability=QoSReliabilityPolicy.RELIABLE,  # 设置可靠性策略为RELIABLE
            history=QoSHistoryPolicy.KEEP_LAST,  # 设置历史策略为KEEP_LAST
            depth=1  # 设置深度为1
        )
        self.publisher_ = self.create_publisher(String, 'topic', qos_profile)  # 创建发布者对象
        timer_period = 0.5  # 设置定时器回调的时间间隔为0.5秒
        self.timer = self.create_timer(timer_period, self.timer_callback)  # 创建定时器，设置回调函数为timer_callback
        self.i = 0

    def timer_callback(self):
        msg = String()  # 创建String消息类型的对象
        msg.data = 'Hello World: %d' % self.i  # 设置消息的数据
        self.publisher_.publish(msg)  # 发布消息
        self.i += 1  # 自增计数器
```

首先创建了一个 MinimalPublisher 节点类,用于实现 ROS2 可靠模式下的循环发布。在构造函数__init__()中,首先创建了一个 QosProfile 对象,将发布质量配置为可靠模式;然后通过 QosProfile 对创建的发布者对象 publisher_进行了初始化。构造函数还设置了定时器周期 0.5s,并创建定时器对象。并将计数器 i 初始化为 0。定义了定时器回调函数 timer_callback,在函数体中,首先构造了 String 类型的消息对象 msg,并利用字符串格式化将当前计数 i 值写入消息内容。然后通过之前定义的 publisher_对象发布这条消息。最后计数 i 增加 1。

7.2 话题订阅

根据实现的效果，梳理程序的过程逻辑，如下图所示：



创建了一个 MinimalSubscriber 节点类,用于实现 ROS2 下的可靠模式订阅。在构造函数中,它首先使用 QosProfile 对象配置订阅质量,将可靠性设置为 RELIABLE,保留历史记录的最近一条消息,并将缓冲区 depth 设置为 1。然后通过 QosProfile 对象来创建订阅者对象,订阅 topic 并指定回调函数。回调函数 listener_callback 只是简单打印接收到的消息内容。主函数用于初始化节点,创建订阅者实例,进入主循环驱动回调函数,和释放节点资源。通过 QosProfile 的配置,可以保证订阅者在回调中能获得最近一条正确的消息。

- 主函数

```
def main(args=None):
    rclpy.init(args=args)  # 初始化ROS节点

    minimal_subscriber = MinimalSubscriber()  # 创建MinimalSubscriber对象

    rclpy.spin(minimal_subscriber)  # 进入主循环

    minimal_subscriber.destroy_node()  # 销毁节点
    rclpy.shutdown()  # 关闭ROS
```

首先调用 rclpy.init()对 ROS2 Python 接口进行初始化，然后实例化 MinimalSublisher()，然后在 ROS2 节点的事件循环执行 minimal_sublisher。

◆ MinimalSubscriber 类

```
class MinimalSubscriber(Node):  # 定义一个继承自Node类的MinimalSubscriber类

    def __init__(self):
        super().__init__('minimal_subscriber')  # 调用父类构造函数初始化节点
        qos_profile = QoSProfile(  # 创建QoSProfile对象
            reliability=QoSReliabilityPolicy.RELIABLE,  # 设置可靠性策略为RELIABLE
            history=QoSHistoryPolicy.KEEP_LAST,  # 设置历史策略为KEEP_LAST
            depth=1  # 设置深度为1
        )
        self.subscription = self.create_subscription(String, 'topic', self.listener_callback, qos_profile)  # 创建订阅者对
象，并设置回调函数为listener_callback

    def listener_callback(self, msg):
        self.get_logger().info('I heard: "%s"' % msg.data)  # 打印接收到的消息内容
```

首先创建了一个 MinimalSubscriber 节点类,用于实现 ROS2 下的可靠订阅功能。它在构造函数__init__()中,首先创建了 QosProfile 对象,将订阅质量配置为可靠模式(RELIABLE、KEEP_LAST 等)。然后通过 QosProfile 对象和指定的回调函数,创建了订阅对象 subscription,订阅 topic 和指定回调函数。定义了订阅回调函数 listener_callback,这个回调函数仅仅是打印接收到的消息内容 msg。